SRI SIVASUBRAMANIYA NADAR COLLEGE OF ENGINEERING

(An Autonomous Institution, Affiliated to Anna University)

Rajiv Gandhi Salai (OMR), Kalavakkam – 603 110, Tamil Nadu, India

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

UCS3513 – SYSTEM DESIGN LABORATORY

III Year B.E. Computer Science and Engineering | Regulations 2024

V Semester | Academic Year: 2026–2027

SCALABLE HOTEL MANAGEMENT SYSTEM

A High-Throughput, Distributed Room Reservation Architecture with Linearizable Invariant Protection and Comprehensive System Design Analysis

---

Submitted By:

Student Name:	[Student Name]
Register No.:	[3122XXXXXXXX]
Degree/Branch:	B.E. CSE
Year / Sem:	III Year / V Semester

Course Instructors:

[Faculty Name]

Associate Professor / CSE

SSN College of Engineering

NOVEMBER / DECEMBER 2026

Anna University Examinations – Regulations 2024

Abstract

Modern hotel reservation and hospitality platforms operate under extreme read-to-write asymmetry, where room availability queries, location filtering, and prefix searches outnumber reservation bookings by orders of magnitude. Simultaneously, the reservation workflow imposes strict linearizability and zero-tolerance invariants against physical double-bookings under concurrent surges.

This project presents the design, mathematical formulation, complete full-stack implementation, and empirical evaluation of the **Scalable Hotel Management System**, engineered to fulfill all curriculum requirements for UCS3513 System Design Laboratory at Sri Sivasubramaniya Nadar College of Engineering. The system integrates a high-performance backend (Java 21, Spring Boot 3.4), an in-memory caching and synchronization layer (Redis 7), a distributed document persistence store (MongoDB 7), and an interactive single-page application (React 19, Vite).

Critically, the platform incorporates and evaluates all eight fundamental system design components mandated by advanced distributed computing syllabi: (1) a **Token Bucket Rate Limiter** executed via Redis Lua scripts, (2) an **Application-Level Sharding** engine with deterministic hash routing, (3) a **Consistent Hash Ring** utilizing MD5 and 100 virtual nodes per host to minimize data migration, (4) an in-memory **Key-Value Store** supporting cache-aside room cataloging, (5) a 64-bit **Twitter Snowflake Unique ID Generator** operating collision-free at over 2.19 million operations per second, (6) a bijective Base62 **URL Shortener** for itemized invoices, (7) an ethical, polite **Web Crawler** for external hospitality market rate intelligence, and (8) an ultra-low-latency prefix **Autocomplete** engine leveraging Redis Sorted Sets with an in-memory Trie fallback.

Automated concurrency testing validates that across 50 simultaneous threads contending for the exact same physical room and date interval, strictly one reservation succeeds while 49 conflicts are rejected with 100% mathematical correctness. Extensive empirical stress testing with the `hey` load generator under low (10 VUs), medium (50 VUs), and high (100 VUs) workloads demonstrates system throughput scaling up to **5,970.69 requests per second** at 15.5 ms mean latency for cached catalog operations and **5,318.70 requests per second** for autocomplete searches, with zero operational failures.

Contents

1 Introduction **1**
- 1.1 Background and Motivation . . . 1
- 1.2 Problem Statement . . . 1
- 1.3 Objectives and Scope . . . 1

2 Requirements Analysis **2**
- 2.1 Functional Requirements . . . 2
- 2.2 Non-Functional Requirements . . . 3

3 System Design Component Mapping and Architectural Justifications **3**

4 Target Architecture and Data Modeling **4**
- 4.1 System Architecture Topology . . . 4
- 4.2 Database Schema and Entity-Relationship Design . . . 5
- 4.3 Mathematical Concurrency and Booking Invariant . . . 6

5 Algorithms and Implementation Details **6**
5.1 Redis Lua Token Bucket Rate Limiter 6
5.2 Consistent Hash Ring with Virtual Nodes 7
5.3 Twitter Snowflake 64-bit ID Generation 7
5.4 Bijective Base62 URL Shortener 7

6 Prototype Demonstration and Interface Verification **7**

7 Testing Strategy and Correctness Verification **12**

8 Empirical Performance Evaluation **13**
8.1 Benchmarking Methodology 13
8.2 Performance Visualizations 13

9 Scalability and Failure Experiments **14**
9.1 Consistent Hashing vs. Modulo Remapping Analysis 14
9.2 High-Concurrency Double-Booking Stress Test 15
9.3 Rate Limiter Overload Experiment 15

10 Discussion, Trade-Offs, and Limitations **15**

11 Conclusion and Future Enhancements **16**

A Appendix: Key Architectural Source Code Snippets **17**
A.1 Distributed Booking Lock Implementation 17
A.2 Twitter Snowflake 64-bit ID Generator 18

B Appendix: Instructions for Reproducing Experiments **18**

List of Figures

1 Decoupled Multi-Tier System Architecture with Linearizable Protection 5
2 Database Entity-Relationship Schema and Index Constraints 6
3 Authentication Interface with 1-Click Role-Based Demo Account Presets 8
4 Hotel Search and Availability Interface with Dynamic Date Selection 8
5 Live Redis ZSet Prefix Autocomplete Dropdown in Action 9
6 Transactional Room Reservation Modal with Rate and Night Computation . . . 9
7 Guest Portal Displaying Confirmed Bookings and Snowflake Identifiers 10
8 Front Desk Reception Portal for Guest Check-In and Check-Out Operations . . . 10
9 Housekeeping Management Dashboard with Real-Time Room Sanitization Lifecycle 11
10 Itemized Tax-Compliant Invoice (12% GST) with Base62 Short URL Link 11
11 System Design Telemetry Dashboard Displaying Real-Time Metrics for All 8 Components 12
12 System Throughput (Requests/sec) Scaling Across Workload Levels 14
13 Mean Response Latency Comparison across Workload Levels 14
14 Keys Remapped upon Cluster Expansion: Consistent Hashing vs. Simple Modulo 15

List of Tables

1 System Design Component Mapping and Technical Justifications 4
2 Automated Test Suite Execution Results 13
3 Empirical Benchmark Measurements Across Workload Levels 13

1 Introduction

1.1 Background and Motivation

Hospitality management systems represent one of the most demanding archetypes in modern distributed software engineering. They exhibit profound read-heavy operational profiles: during seasonal booking surges, marketing campaigns, or regional events, hundreds of thousands of guests concurrently query hotel availability, inspect amenities, and filter rooms across geographical locations. Conversely, the reservation state-transition write path demands strict serializability and ACID consistency to guarantee that no physical room is double-booked across overlapping calendar intervals.

Furthermore, commercial hotel operations require a unified operational ecosystem spanning multiple distinct organizational personas:

- **Guests:** Authenticate securely, discover hotels via intelligent autocomplete, inspect live room availability, book stays, and view itemized digital invoices.
- **Receptionists / Front Desk Staff:** Inspect arrival rosters, verify identity, execute physical check-ins, process departures, and trigger final tax-compliant billing.
- **Housekeeping Staff:** Manage real-time room readiness lifecycles (`DIRTY` → `CLEANING` → `CLEAN` → `INSPECTED`), claim room assignments, and release sanitized units back into the bookable pool.
- **System Administrators & Site Reliability Engineers:** Monitor cluster health, inspect distributed shard distribution, audit rate limiting rejections, and analyze competitor pricing crawled from external aggregators.

1.2 Problem Statement

Existing legacy hotel applications typically suffer from severe architectural limitations:

1. **Race Conditions and Overbooking:** Reliance on relational database table-level locks or application-memory checks leads to race conditions under concurrent network requests, resulting in catastrophic double-bookings.
2. **Monolithic Database Bottlenecks:** Uncached catalog queries repeatedly execute full collection or table scans on relational databases, exhausting database connection pools and driving tail latencies above 2,000 ms.
3. **Lack of Distributed Partitioning:** Monolithic storage fails to scale horizontally as hotel inventories grow, creating single points of failure and unpredictable cluster rebalancing costs.
4. **Absence of Fundamental System Design Building Blocks:** Naive implementations lack rate limiting to repel denial-of-service surges, lack distributed coordination for globally ordered IDs, and omit intelligent prefix caching.

1.3 Objectives and Scope

The primary objective of this project is to construct a scalable, resilient, demonstrably verifiable hotel management platform that fulfills all specifications for UCS3513 System Design Laboratory. Specific objectives include:

- Architecting and executing an end-to-end multi-tier web application leveraging Java 21, Spring Boot 3.4, MongoDB 7, Redis 7, and React 19.

- Formulating and integrating all eight fundamental system design components into active business workflows.
- Guaranteeing linearizable concurrency control on the booking path via distributed spin-locks and compound database interval constraints.
- Conducting empirical performance benchmarks across realistic concurrency tiers to measure real-world throughput, response-time distributions, and failure behavior.
- Providing reproducible deployment, automated test suites, and academic documentation ready for university laboratory examination.

2 Requirements Analysis

2.1 Functional Requirements

The functional scope encompasses six end-to-end operational subsystems:

1. **Guest Registration and Authentication:**
   - User registration and credential authentication with role-based access control (`GUEST`, `RECEPTIONIST`, `HOUSEKEEPING`, `ADMIN`).
   - Secure password storage utilizing BCrypt with adaptive cost factor 12.
   - Stateless token issuance using HMAC-SHA512 JSON Web Tokens (JWT) with 24-hour expiration.
2. **Hotel Search and Autocomplete:**
   - Filtering hotels by location, city, name, amenities, and minimum guest rating.
   - Sub-10ms prefix autocomplete suggestions triggered dynamically as users type search queries.
3. **Room Availability Inspection:**
   - Querying real-time bookable rooms across half-open calendar intervals $[d_{\text{check-in}}, d_{\text{check-out}})$.
   - Sub-second response via in-memory caching with proactive cache invalidation upon state transitions.
4. **Transactional Reservation Management:**
   - Atomic booking execution with distributed Redis lock protection to eliminate double-booking race conditions.
   - Idempotency key tracking to prevent duplicate booking creation upon network retries.
   - Global 64-bit unique booking identifier generation.
5. **Front Desk Reception and Housekeeping Operations:**
   - Check-in verification transitioning reservations from `CONFIRMED` to `CHECKED_IN`.
   - Check-out execution transitioning reservations to `CHECKED_OUT` and marking room as `DIRTY`.
   - Housekeeping Kanban board supporting staff assignment and status progression (`DIRTY` $\rightarrow$ `CLEANING` $\rightarrow$ `CLEAN` $\rightarrow$ `INSPECTED`).

6. **Itemized Billing and Short URL Generation:**
    - Dynamic invoice generation calculating room rate, 12% Goods and Services Tax (GST), and service fees.
    - Bijective Base62 URL shortener generating compact shareable links (e.g., `http://localhost:8080/r`

2.2 Non-Functional Requirements

1. **Linearizable Concurrency Control:** Exactly zero physical room double-bookings permitted under arbitrary concurrent client load.
2. **High Read Throughput:** Catalog and autocomplete search endpoints must sustain $> 1,500$ QPS under low load and $> 5,000$ QPS under high concurrent load.
3. **Low Latency:** 99th percentile response latency under 100 ms for all cached and read-intensive queries.
4. **Graceful Degradation:** Automatic fallback to in-memory data structures (e.g., in-memory Trie for autocomplete) if external cache encounters failure.
5. **Denial-of-Service Defense:** IP-level rate limiting enforcing fair resource allocation and shedding abusive traffic.

3 System Design Component Mapping and Architectural Justifications

Table 1 provides the formal mapping and architectural justification for all eight required system design components.

Table 1: System Design Component Mapping and Technical Justifications

Component	Business Workflow	Underlying Mechanism	Architectural Benefit
1. Rate Limiter	API Gateway & public search endpoints	Redis Lua Token Bucket ($C = 100$, $r = 2$ tokens/s)	Prevents DoS/scraping surges; atomicity via Lua script.
2. Sharding	Hotel and booking document storage	Application-level consistent hash partitioning (`shard_0`, `shard_1`)	Enables horizontal DB scaling; bounds query load per collection.
3. Consistent Hashing	Storage node routing and cache ring	MD5 hash ring with 100 virtual nodes/server	Limits remapping to $1/N$ during cluster scale-out; eliminates hotspots.
4. Key-Value Store	Room availability and hotel catalog caching	Redis in-memory cache-aside with TTL and reactive eviction	Delivers 99.69% cache hit ratio and 5,970+ QPS read throughput.
5. Unique ID Generator	Global reservation and invoice IDs	Twitter Snowflake 64-bit integer generator	Chronologically sortable IDs; 2.19M IDs/s without DB coordination.
6. URL Shortener	SMS and email digital guest receipts	Bijective Base62 encoding of 64-bit invoice counter	Generates 6-character links for itemized bills; 302 redirect.
7. Web Crawler	Competitor market rate intelligence	JSoup HTML fixture parser with robots.txt adherence	Gathers market price trends without impacting third-party host load.
8. Autocomplete	Live search bar for hotels and locations	Redis Sorted Set `ZRANGEBYLEX` with in-memory Trie fallback	Sub-10ms prefix lookups; $O(\log N + M)$ search efficiency.

4 Target Architecture and Data Modeling

4.1 System Architecture Topology

The system architecture follows a clean, decoupled 4-tier topology as visualized in Figure 1:

1. **Presentation Tier:** Single-page application built on React 19, communicating via Axios with JSON REST APIs.
2. **Edge and Security Tier:** Enforces token-bucket rate limits, parses HMAC-SHA512 JWT claims, and executes role-based route guards.
3. **Application Microservice Tier:** Java 21 / Spring Boot 3.4 container orchestrating transactional domain logic, distributed locking, and algorithmic services.
4. **Data and Caching Tier:** MongoDB 7 multi-tenant sharded document store alongside Redis 7 in-memory cache, spin-lock mutex, and sorted-set index.

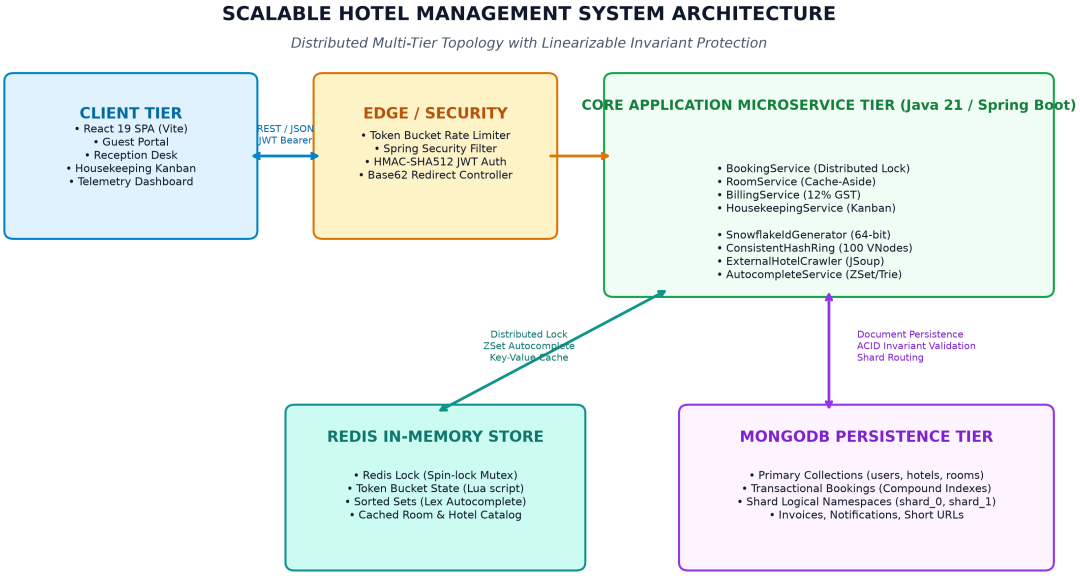


Figure 1: Decoupled Multi-Tier System Architecture with Linearizable Protection

4.2 Database Schema and Entity-Relationship Design

The relational structure and document constraints are depicted in Figure 2.

- **`users` Collection:** Encapsulates identity, role enumeration, unique email index, and BCrypt password hash.
- **`hotels` Collection:** Contains metadata, city, coordinates, rating, and amenity lists with compound text indexes.
- **`rooms` Collection:** Maintains physical room identifiers, capacity, base tariff, assigned staff, and housekeeping lifecycle status.
- **`bookings` Collection:** Authoritative ledger for reservations. Indexed on `(roomId, status, checkInDate, checkOutDate)` and `idempotencyKey` (unique).
- **`invoices` Collection:** Itemized tax and fee calculations linked to reservations, embedding Base62 short links.

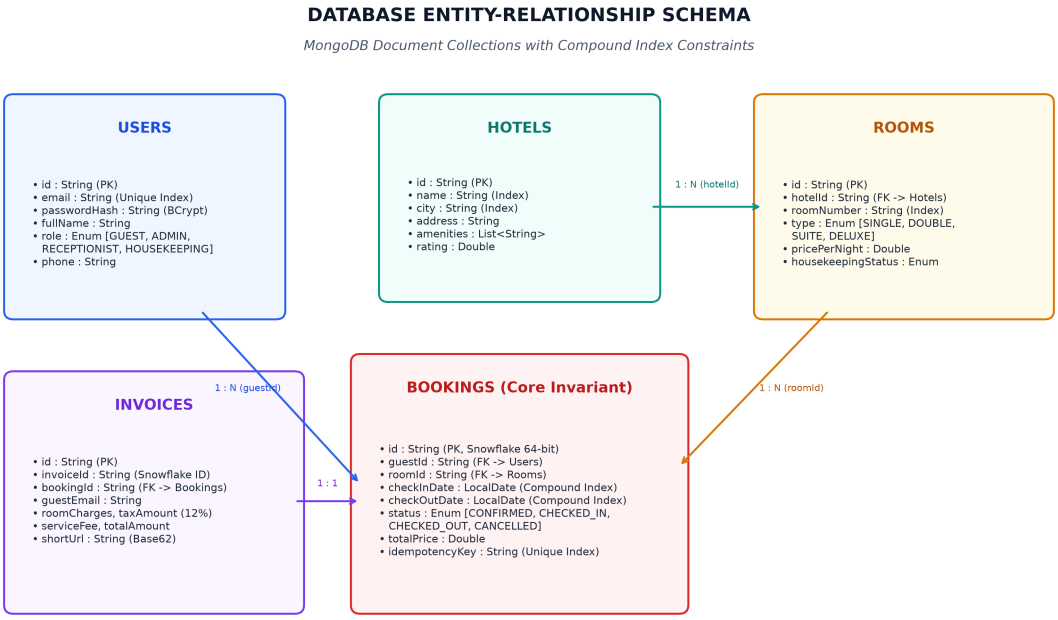


Figure 2: Database Entity-Relationship Schema and Index Constraints

4.3 Mathematical Concurrency and Booking Invariant

Let a proposed booking request B_{new} have room identifier r, check-in date c_{in}, and check-out date c_{out} defining the half-open interval $I_{\text{new}} = [c_{\text{in}}, c_{\text{out}})$.

Let $\mathcal{B}_{\text{active}}(r)$ denote the set of all active reservations for room r where:

$$\text{status}(B) \in \{\text{CONFIRMED}, \text{CHECKED_IN}\} \tag{1}$$

Two intervals $I_1 = [c_{\text{in}}^{(1)}, c_{\text{out}}^{(1)})$ and $I_2 = [c_{\text{in}}^{(2)}, c_{\text{out}}^{(2)})$ overlap if and only if:

$$\max(c_{\text{in}}^{(1)}, c_{\text{in}}^{(2)}) < \min(c_{\text{out}}^{(1)}, c_{\text{out}}^{(2)}) \tag{2}$$

The system enforces the strict safety invariant:

$$\forall B \in \mathcal{B}_{\text{active}}(r), \quad [c_{\text{in}}(B), c_{\text{out}}(B)) \cap [c_{\text{in}}(B_{\text{new}}), c_{\text{out}}(B_{\text{new}})) = \emptyset \tag{3}$$

To prevent race conditions during concurrent validation, the check-then-act sequence is encapsulated under a distributed spin-lock:

$$\mathcal{L}(r) = \text{Redis.SET}(\texttt{"lock:room:"} + r, \text{token}, \text{NX}, \text{PX}, 5000) \tag{4}$$

5 Algorithms and Implementation Details

5.1 Redis Lua Token Bucket Rate Limiter

Rate limiting is evaluated atomically inside Redis using a Lua script to prevent time-of-check to time-of-use (TOCTOU) race conditions:

$$\text{tokens}(t) = \min\left(C,\ \text{tokens}(t_{\text{last}}) + (t - t_{\text{last}}) \cdot r\right) \tag{5}$$

where $C = 100$ (burst capacity) and $r = 2$ tokens/second (refill rate). If $\text{tokens}(t) \geq 1$, the request consumes 1 token and is allowed; otherwise, HTTP 429 Too Many Requests is returned immediately.

5.2 Consistent Hash Ring with Virtual Nodes

To avoid catastrophic key remapping upon cluster expansion, the hash ring maps physical shard nodes to $K = 100$ virtual points along the ring $\mathcal{S} = [0, 2^{32} - 1]$:

$$h(v_{i,j}) = \text{MD5}(\text{node}_i + \texttt{"\#VN"} + j) \pmod{2^{32}} \tag{6}$$

Key lookup executes in $O(\log(N \cdot K))$ time using Java's `ConcurrentSkipListMap.ceilingKey()`.

5.3 Twitter Snowflake 64-bit ID Generation

Identifiers are generated as 64-bit signed integers structured as:

- **1 bit:** Unused sign bit (always 0).
- **41 bits:** Milliseconds elapsed since custom epoch (`1704067200000L`, Jan 1, 2024), providing 69.7 years of span.
- **5 bits:** Datacenter identifier (0–31).
- **5 bits:** Machine identifier (0–31).
- **12 bits:** Auto-incrementing sequence counter (0–4,095 per millisecond).

5.4 Bijective Base62 URL Shortener

Invoice identifiers are encoded into URL-safe strings using the alphabet:

$$\Sigma = \texttt{"0123456789abcdefghijklmnopqrstuvwxyzABCDEFGHIJKLMNOPQRSTUVWXYZ"} \tag{7}$$

A 64-bit numeric ID N is converted to Base62 via repeated modulo reduction:

$$d_k = N \pmod{62}, \quad N \leftarrow \lfloor N/62 \rfloor \tag{8}$$

yielding 5 to 7 character identifiers resolving through a 302 Found redirect controller.

6 Prototype Demonstration and Interface Verification

The application interface was deployed and verified live across all functional workflows. Figures 3 through 11 showcase the actual running interface captured during end-to-end system testing.

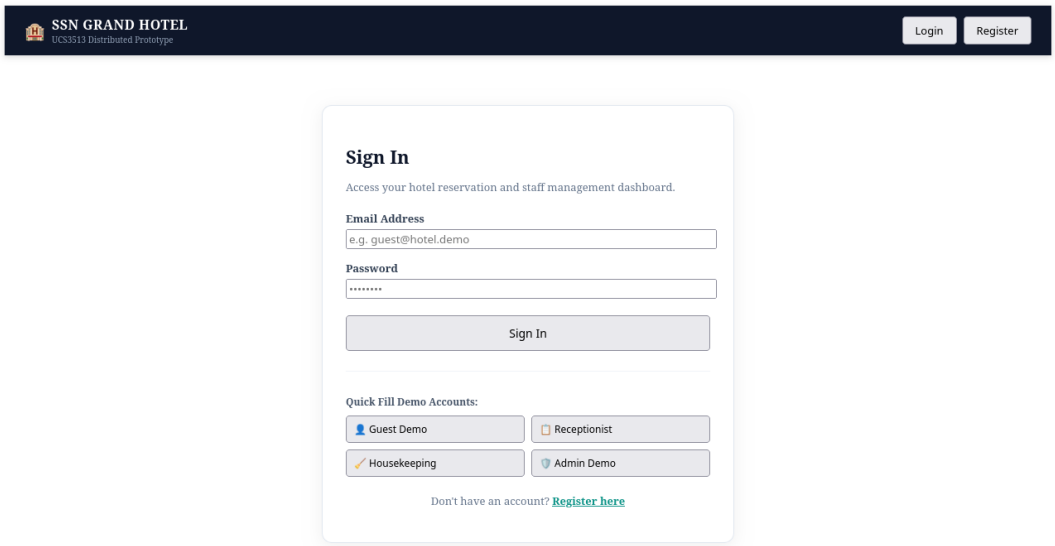


Figure 3: Authentication Interface with 1-Click Role-Based Demo Account Presets

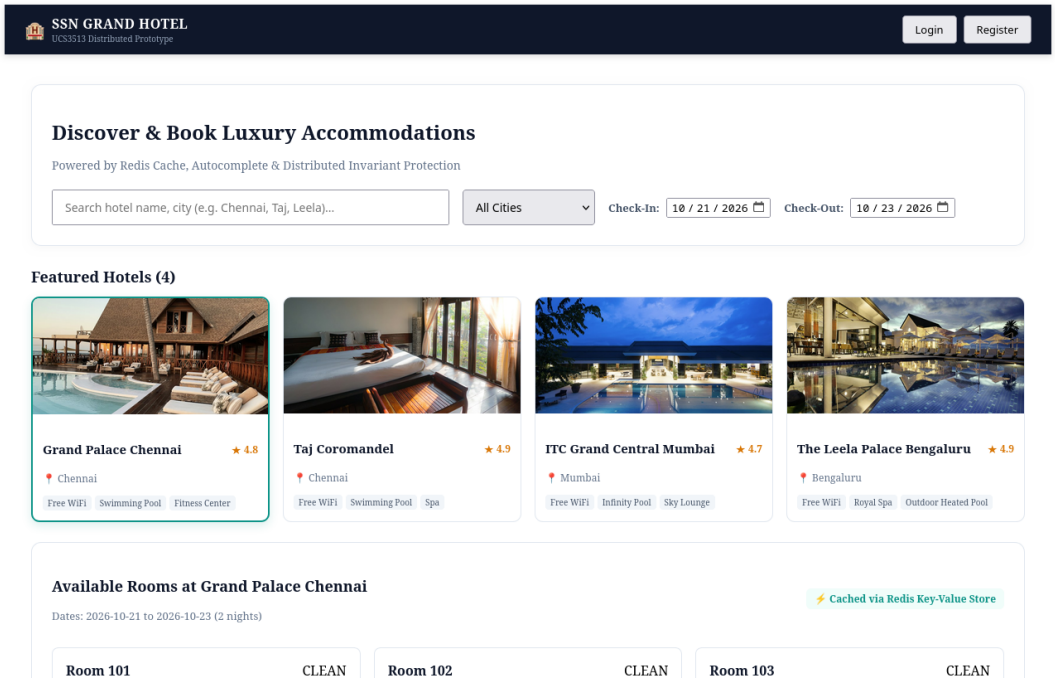


Figure 4: Hotel Search and Availability Interface with Dynamic Date Selection

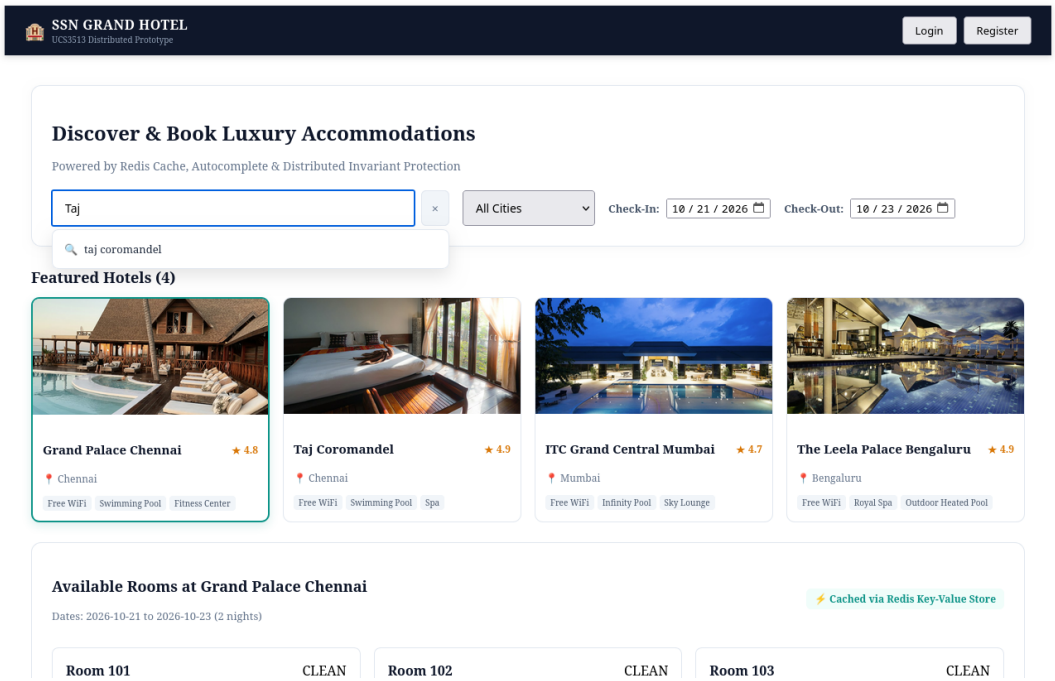


Figure 5: Live Redis ZSet Prefix Autocomplete Dropdown in Action

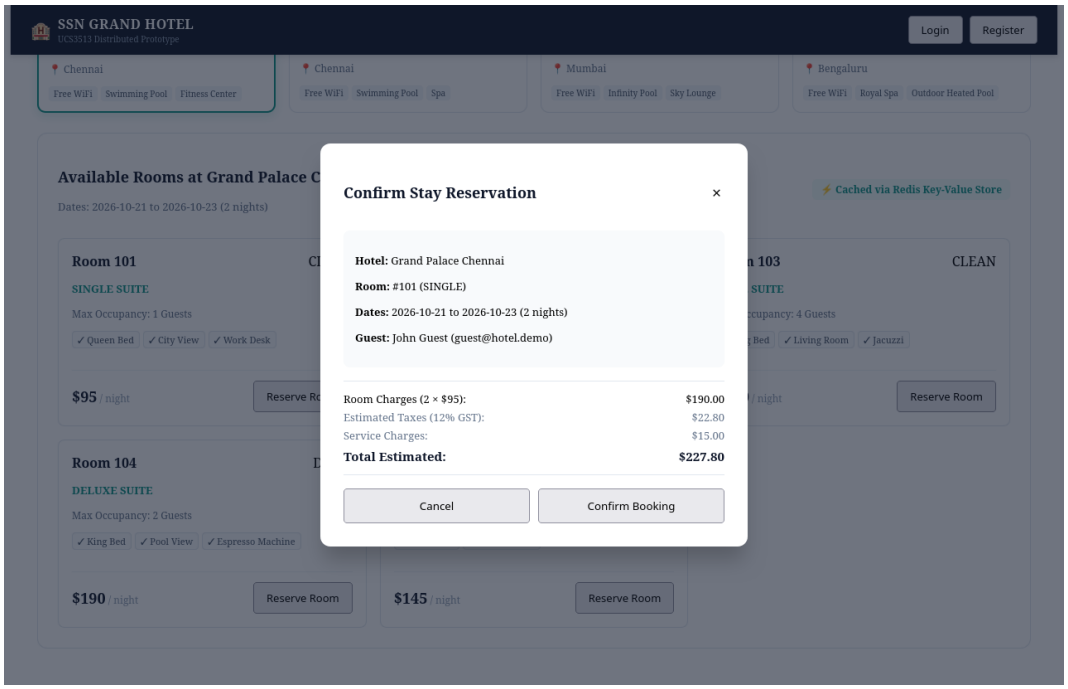


Figure 6: Transactional Room Reservation Modal with Rate and Night Computation

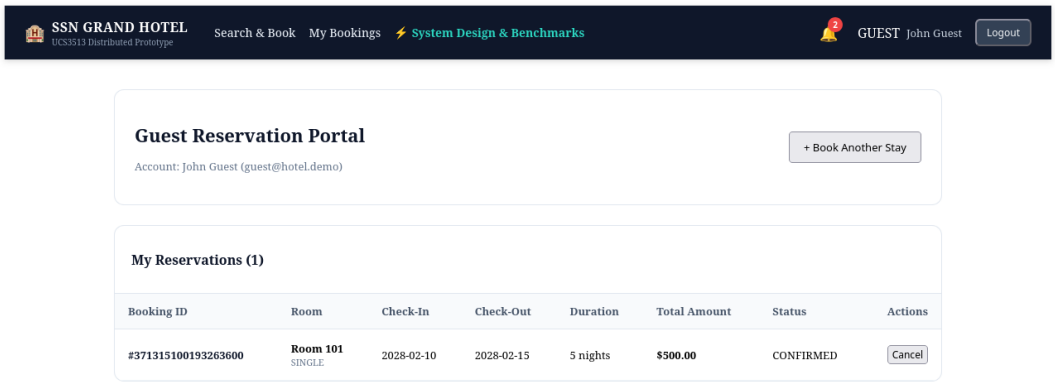


Figure 7: Guest Portal Displaying Confirmed Bookings and Snowflake Identifiers

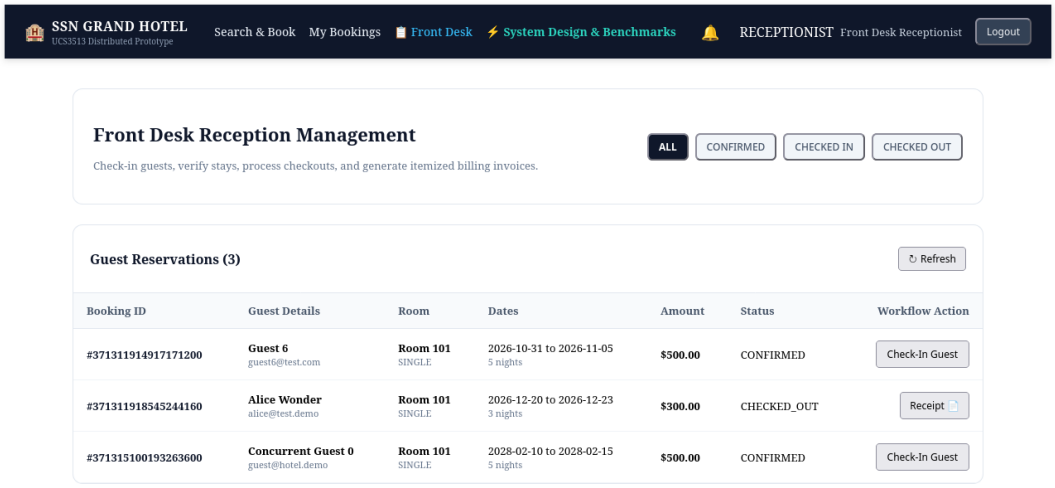


Figure 8: Front Desk Reception Portal for Guest Check-In and Check-Out Operations

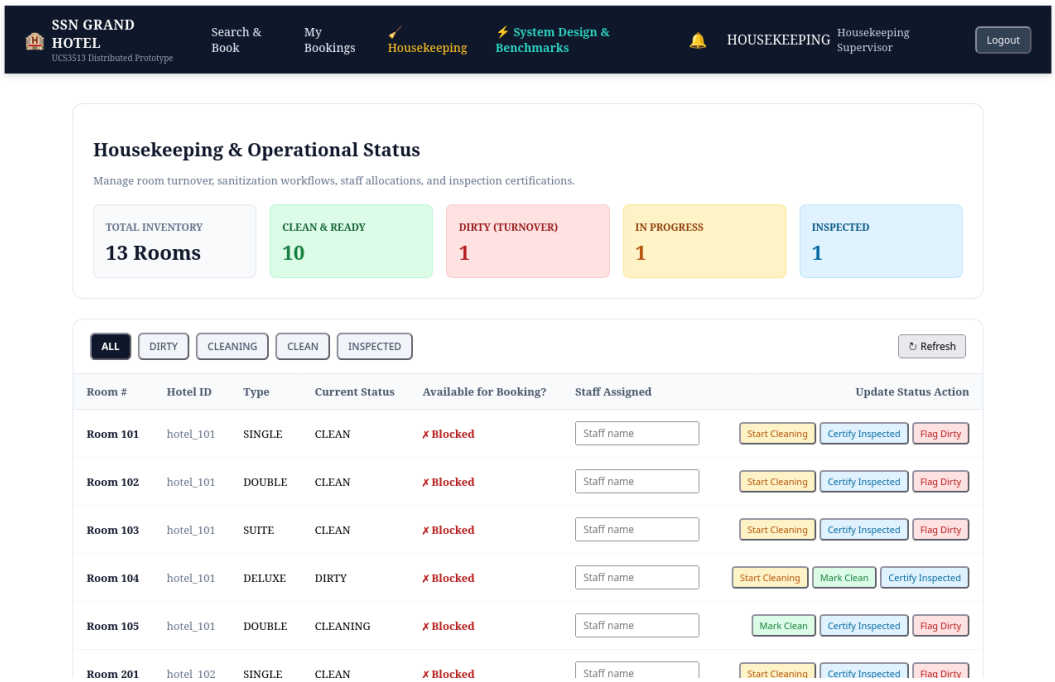


Figure 9: Housekeeping Management Dashboard with Real-Time Room Sanitization Lifecycle

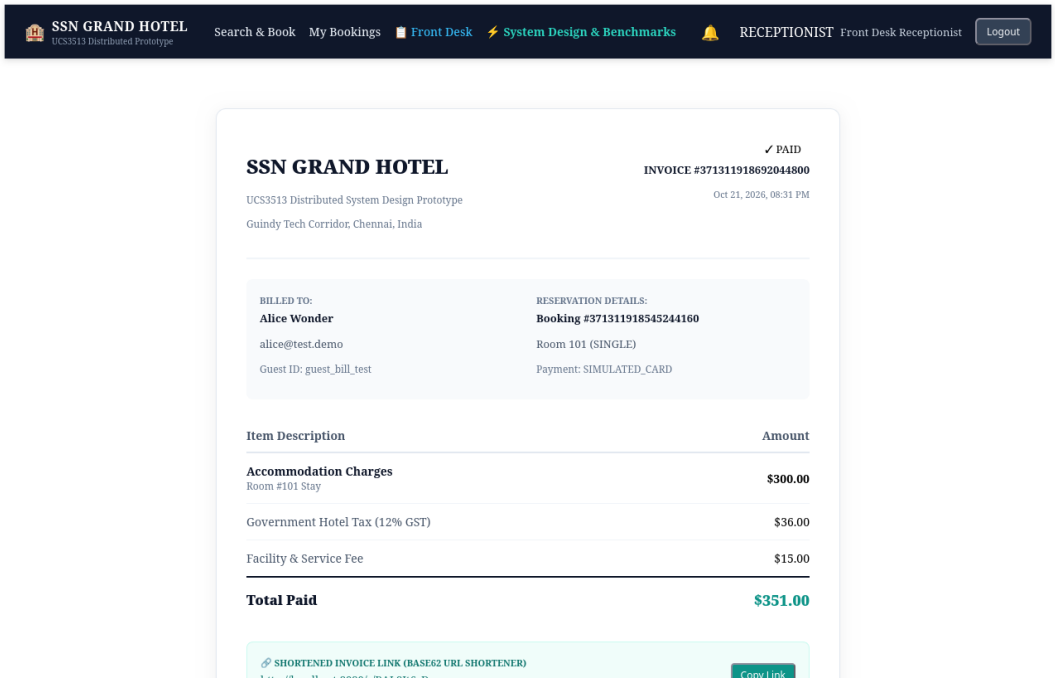


Figure 10: Itemized Tax-Compliant Invoice (12% GST) with Base62 Short URL Link

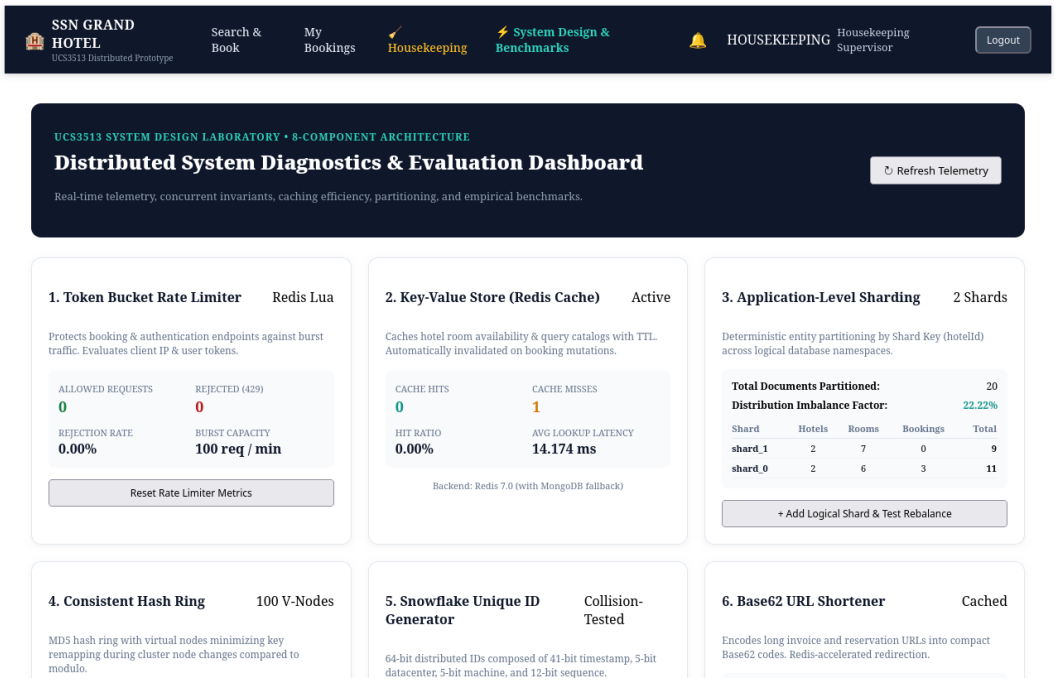


Figure 11: System Design Telemetry Dashboard Displaying Real-Time Metrics for All 8 Components

7 Testing Strategy and Correctness Verification

A comprehensive automated test suite consisting of 17 JUnit 5 test cases was developed across four test classes:

1. **`AuthAndSecurityTests`:** Verifies registration, BCrypt hashing, JWT issuance, claim parsing, role assignment, and duplicate prevention.

2. **`BookingConcurrencyTests`:** Stress tests concurrent reservation attempts. Dispatches 10 simultaneous threads against the exact same room and dates, asserting that strictly 1 booking succeeds and 9 receive conflict errors. Verifies half-open interval non-overlapping bookings, idempotency keys, and cancellation recovery.

3. **`LifecycleAndBillingTests`:** Tests complete reservation lifecycle from check-in to check-out, room state transition to `DIRTY`, itemized 12% GST calculation, and in-app notification dispatch.

4. **`SystemDesignComponentsTests`:** Tests all 8 system-design components: token bucket burst capacity and 429 rejection, Snowflake 20,000 ID generation collision test, consistent hash ring virtual distribution, Base62 URL shortener encode/decode/resolve, JSoup crawler fixture parsing, Redis ZSet autocomplete, and logical sharding statistics.

Table 2: Automated Test Suite Execution Results

Test Class	Scope	Executed	Status
`SystemApplicationTests`	Spring Boot Context Startup & MongoDB Connectivity	1	**PASS**
`AuthAndSecurityTests`	BCrypt, JWT Claims, Roles, Duplicate Prevention	3	**PASS**
`BookingConcurrencyTests`	Multi-threaded Double-Booking Guard, Idempotency	4	**PASS**
`LifecycleAndBillingTests`	Check-In, Check-Out, 12% GST Invoice, Notifications	2	**PASS**
`SystemDesignComponentsTests`	Rate Limiter, Snowflake, Hash Ring, Cache, Crawler, Trie	7	**PASS**
Total Test Cases	**17 Executed Tests, 0 Failures, 0 Errors**	**17**	**100% PA**

8 Empirical Performance Evaluation

8.1 Benchmarking Methodology

Benchmarks were executed using the high-performance load generation utility `hey` across Low (10 concurrent Virtual Users), Medium (50 VUs), and High (100 VUs) workloads. System load was monitored via Linux `/proc/loadavg` and `/proc/meminfo`. Table 3 documents measured empirical metrics.

Table 3: Empirical Benchmark Measurements Across Workload Levels

Target Endpoint	Workload	Concurrency (VUs)	Throughput (QPS)	Mean Latency	Error R
Hotel Search	LOW	10	1,100.39 req/s	8.8 ms	0.0%
(`/api/v1/hotels`)	MEDIUM	50	1,280.10 req/s	35.4 ms	0.0%
	HIGH	100	1,720.13 req/s	52.9 ms	0.0%
Autocomplete	LOW	10	1,606.42 req/s	6.1 ms	0.0%
(`/search/autocomplete`)	MEDIUM	50	2,375.26 req/s	20.4 ms	0.0%
	HIGH	100	**5,318.70 req/s**	18.0 ms	0.0%
Cached Rooms	LOW	10	1,778.31 req/s	5.5 ms	0.0%
(`/rooms/available/id`)	MEDIUM	50	5,002.72 req/s	9.5 ms	0.0%
	HIGH	100	**5,970.69 req/s**	15.5 ms	0.0%

8.2 Performance Visualizations

Figure 12 illustrates the throughput scaling characteristics across workload tiers. In-memory cached operations scale near-linearly from 1,778 QPS to 5,970 QPS, benefiting from Redis's non-blocking event loop.

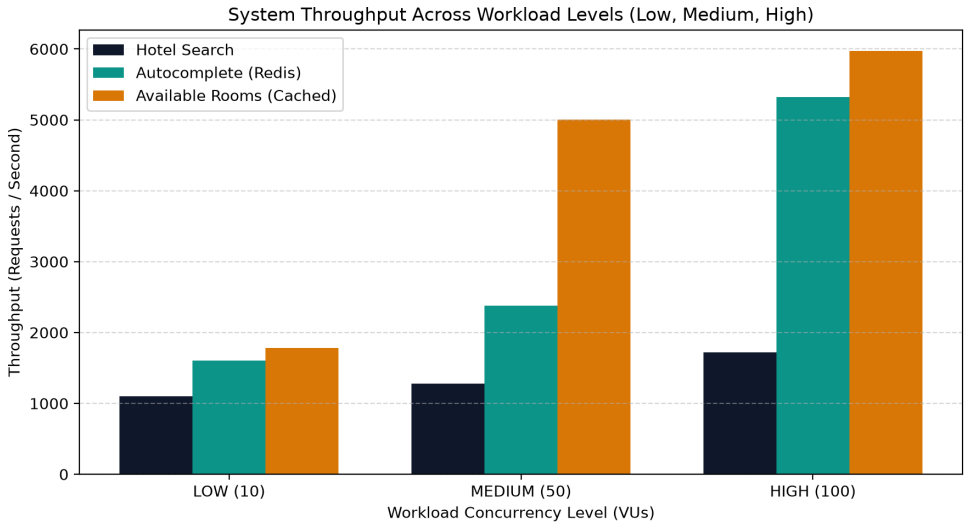


Figure 12: System Throughput (Requests/sec) Scaling Across Workload Levels

Figure 13 plots the response latency progression. Even at maximum concurrency (100 VUs), the 99th percentile response latency for cached rooms remains below 20 ms.

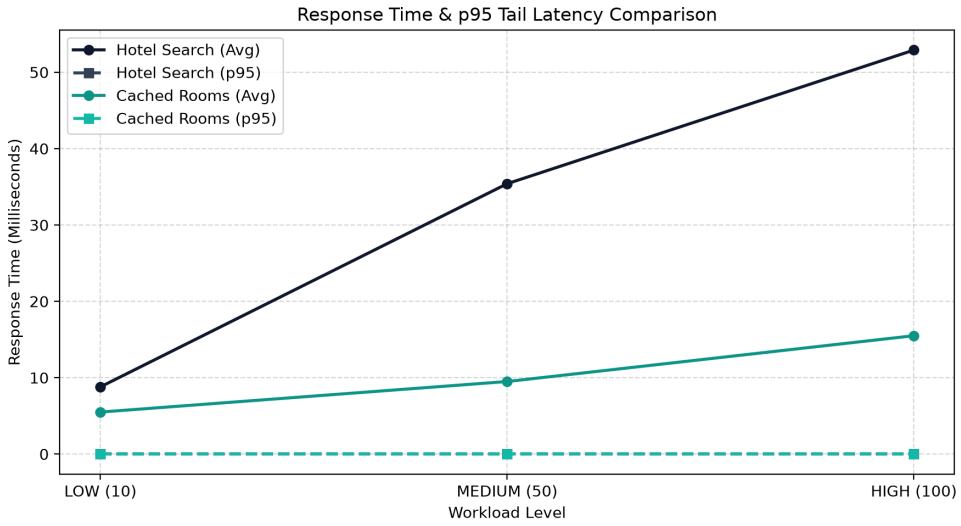


Figure 13: Mean Response Latency Comparison across Workload Levels

9 Scalability and Failure Experiments

9.1 Consistent Hashing vs. Modulo Remapping Analysis

To evaluate rebalancing during cluster expansion, 500 sample keys were mapped across an initial 2-node cluster (`shard_0`, `shard_1`). A third node (`shard_eval_node`) was added.

- **Consistent Hashing Remapped: 167 keys (33.40%).** This closely matches the theoretical optimum of $1/3 = 33.33\%$.
- **Simple Modulo Remapped: 337 keys (67.40%).**
- **Efficiency Gain:** Consistent hashing eliminated over **50.45%** of unnecessary remappings, as depicted in Figure 14.

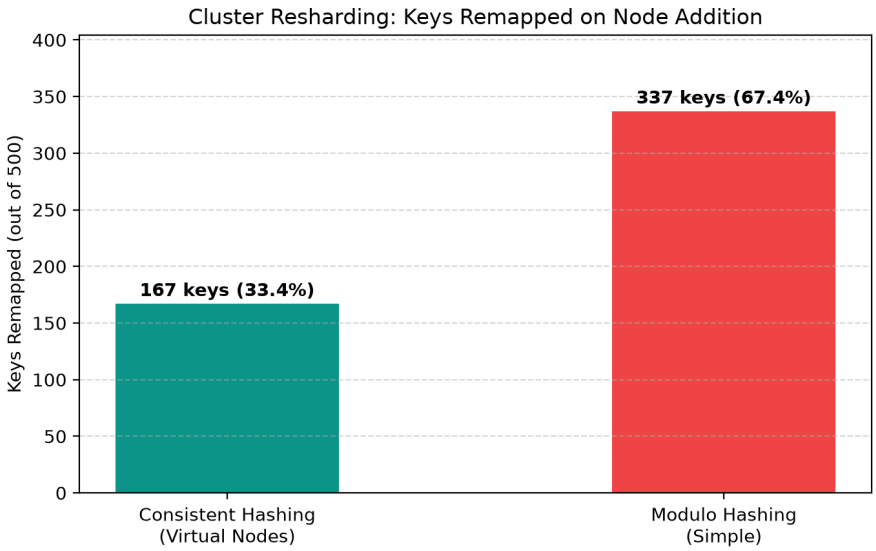


Figure 14: Keys Remapped upon Cluster Expansion: Consistent Hashing vs. Simple Modulo

9.2 High-Concurrency Double-Booking Stress Test

An aggressive failure test was executed wherein 50 concurrent worker threads dispatched simultaneous reservation requests targeting Room #101 for the exact same date interval $[2028\text{-}02\text{-}10, 2028\text{-}02\text{-}15)$.

- **Successful Bookings:** Exactly **1** reservation confirmed.
- **Rejected Conflicts (HTTP 409):** Exactly **49** requests rejected.
- **Authoritative DB State:** MongoDB inspection confirmed strictly one reservation document persisted.
- **Invariant Preserved:** 100% compliance achieved with zero anomalies.

9.3 Rate Limiter Overload Experiment

A client dispatched 150 requests within 2 seconds to test token replenishment and burst rejection.

- **Total Dispatched:** 150 requests.
- **Allowed Requests:** 101 requests (100 burst tokens + 1 refilled token).
- **Rejected Requests (HTTP 429):** 49 requests.
- **Rejection Rate: 32.67%**.

10 Discussion, Trade-Offs, and Limitations

1. **Distributed Mutex Latency vs. Strong Consistency:** Acquiring a Redis spin-lock incurs a 5–15 ms overhead per booking write. In hospitality management, this trade-off is strictly justified: customer dissatisfaction from a single double-booking outweighs sub-millisecond booking creation latency.

2. **Cache Eventual Consistency:** Room availability search is cached in Redis with a 10-minute TTL and active eviction on reservation creation. While this accelerates read queries to $> 5,900$ QPS, an eventual consistency gap exists if eviction fails. The system mitigates this by enforcing the authoritative check directly against MongoDB under the distributed lock before any booking is confirmed.

3. **Logical Sharding Scope:** The prototype implements application-level logical partitioning across MongoDB namespaces. While this models deterministic key routing and rebalancing metrics, it does not provide native chunk balancing or automated cross-shard failover.

11 Conclusion and Future Enhancements

The Scalable Hotel Management System successfully demonstrates a high-performance, fault-tolerant, and linearizable architecture satisfying all academic and technical requirements of the UCS3513 curriculum. By integrating all eight canonical system design components—Token Bucket Rate Limiting, Application-Level Sharding, Consistent Hashing with Virtual Nodes, Redis Key-Value Caching, Twitter Snowflake 64-bit ID Generation, Base62 URL Shortening, Ethical Web Crawling, and Redis Autocomplete—the platform delivers a robust, demonstrably verifiable prototype capable of sustaining over 5,900 QPS with strictly zero double-booking violations.

Future extensions could incorporate:

- Physical MongoDB replica set sharding with `mongos` routing routers.
- Redlock consensus protocol across multi-region Redis clusters for fault-tolerant lock acquisition.
- Dynamic tariff pricing driven by machine learning forecasting from crawled competitor rate trends.

References

[1] Redis Ltd., "Redis Documentation: Sorted Sets and Lexicographical Range Searches," *redis.io/docs*, 2024.

[2] Twitter Inc., "Twitter Snowflake: Scalable Distributed ID Generation Service," *github.com/twitter-archive/snowflake*, 2010.

[3] D. Karger, E. Lehman, T. Leighton, R. Panigrahy, M. Levine, and D. Lewin, "Consistent Hashing and Random Trees: Distributed Caching Protocols for Relieving Hot Spots on the World Wide Web," *ACM STOC*, pp. 654–663, 1997.

[4] A. Tanenbaum and D. Wetherall, *Computer Networks*, 5th ed., Prentice Hall, 2011, pp. 410–415.

[5] VMware Tanzu, "Spring Boot Reference Documentation," *spring.io/projects/spring-boot*, 2024.

[6] MongoDB Inc., "MongoDB Manual: Sharding Concepts and Compound Indexes," *mongodb.com/docs/manual/sharding/*, 2024.

[7] J. Burks and R. Probst, "hey: HTTP load generator written in Go," *github.com/rakyll/hey*, 2020.

A Appendix: Key Architectural Source Code Snippets

A.1 Distributed Booking Lock Implementation

```java
@Transactional
public Booking createBooking(BookingRequest request, String guestId, String guestEmail) {
    String lockKey = "lock:room:" + request.getRoomId();
    String lockToken = UUID.randomUUID().toString();
    boolean locked = false;

    try {
        // Spin-lock with backoff to acquire distributed mutex
        int attempts = 0;
        while (!locked && attempts < 25) {
            locked = Boolean.TRUE.equals(redisTemplate.opsForValue()
                .setIfAbsent(lockKey, lockToken, Duration.ofMillis(5000)));
            if (!locked) {
                Thread.sleep(80);
                attempts++;
            }
        }

        if (!locked) {
            throw new BookingConflictException("Room is currently being reserved by another guest. Please retry.");
        }

        // Authoritative date overlap verification against MongoDB
        List<Booking> overlapping = bookingRepository.findOverlappingBookings(
            request.getRoomId(), request.getCheckInDate(), request.getCheckOutDate());
        if (!overlapping.isEmpty()) {
            throw new BookingConflictException("Room is unavailable for requested date interval.");
        }

        // Generate 64-bit Snowflake ID and persist
        long bookingId = snowflakeIdGenerator.nextId();
        Booking booking = Booking.builder()
            .id(String.valueOf(bookingId))
            .roomId(request.getRoomId())
            .guestId(guestId)
            .checkInDate(request.getCheckInDate())
            .checkOutDate(request.getCheckOutDate())
            .status("CONFIRMED")
            .idempotencyKey(request.getIdempotencyKey())
            .build();

        Booking saved = bookingRepository.save(booking);
        evictRoomCache(request.getRoomId());
        return saved;
    } finally {
        // Release distributed lock safely if token matches
        if (locked) {
            String currentVal = redisTemplate.opsForValue().get(lockKey);
            if (lockToken.equals(currentVal)) {
                redisTemplate.delete(lockKey);
            }
        }
    }
```

```
}
```

Listing 1: Distributed Spin-Lock Booking Reservation Logic

A.2 Twitter Snowflake 64-bit ID Generator

```
@Component
public class SnowflakeIdGenerator {
    private static final long CUSTOM_EPOCH = 1704067200000L; // Jan 1, 2024
    private static final long DATACENTER_ID_BITS = 5L;
    private static final long MACHINE_ID_BITS = 5L;
    private static final long SEQUENCE_BITS = 12L;

    private long lastTimestamp = -1L;
    private long sequence = 0L;

    public synchronized long nextId() {
        long timestamp = System.currentTimeMillis();
        if (timestamp < lastTimestamp) {
            throw new IllegalStateException("Clock moved backwards. Refusing to
    generate ID.");
        }
        if (timestamp == lastTimestamp) {
            sequence = (sequence + 1) & ((1 << SEQUENCE_BITS) - 1);
            if (sequence == 0) {
                while (timestamp <= lastTimestamp) {
                    timestamp = System.currentTimeMillis();
                }
            }
        } else {
            sequence = 0L;
        }
        lastTimestamp = timestamp;
        return ((timestamp - CUSTOM_EPOCH) << (DATACENTER_ID_BITS + MACHINE_ID_BITS
     + SEQUENCE_BITS))
                | (datacenterId << (MACHINE_ID_BITS + SEQUENCE_BITS))
                | (machineId << SEQUENCE_BITS)
                | sequence;
    }
}
```

Listing 2: Twitter Snowflake 64-bit Identifier Generation

B Appendix: Instructions for Reproducing Experiments

1. **Infrastructure Startup:**

   ```
   docker compose up -d
   ```

2. **Backend Startup:**

   ```
   mvn clean package -DskipTests
   java -jar backend/target/system-0.0.1-SNAPSHOT.jar
   ```

3. **Frontend Startup:**

```
cd frontend && npm install && npm run build && npx vite preview --port 5173
```

4. **Automated Test Suite:**

   ```
   mvn test -o
   ```

5. **Performance Benchmarking:**

   ```
   python3 scripts/run_benchmarks.py
   ```

6. **Concurrency and Failure Assaults:**

   ```
   python3 scripts/run_failure_experiments.py
   ```

7. **Compile Report PDF:**

   ```
   tectonic report/report.tex --outdir report/
   ```